

CS571 Programming Languages

Spring 2026

4. Imperative I: names and bindings



Agenda

1. Names and bindings
2. Storage allocation
3. Heap management (garbage collection)
4. Scopes

Limitations of CFG

- CFG for a simple language

```
Prog -> Decl* Stmt*  
Decl -> int ID; | bool ID;  
Stmt -> ID = Expr;
```

- The following two programs are legal

```
int x;  
y = 5;
```

```
bool b;  
b = 5;
```

Limitations of CFG

- CFG for a simple language

```
Prog -> Decl* Stmt*  
Decl -> int ID; | bool ID;  
Stmt -> ID = Expr;
```

- Need to add meanings to symbols beyond syntactic analysis
 - Need **Context Sensitive** Information

```
int x;  
y = 5;
```

y defined?

b number?

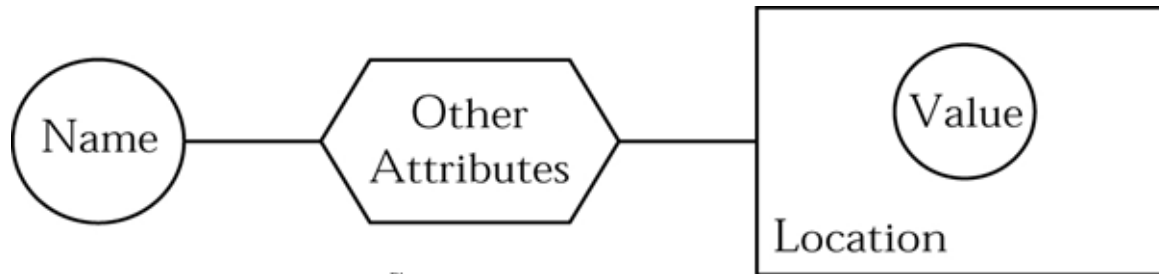
```
bool b;  
b = 5;
```

Names and Binding

- **Name**: a mnemonic string in high-level languages
 - Identifiers in most languages
 - Variables, functions, types, modules (packages)
 - An abstraction of low-level representation, such as memory address and register
- **Binding**: an association between a name and the entity/object it denotes

Variables

- A binding of a name to a memory address
- A variable usually has many attributes: name, address, type, value, lifetime, scope



© 2003 Brooks/Cole - Thomson Learning™

Box-and-circle diagram

Variables

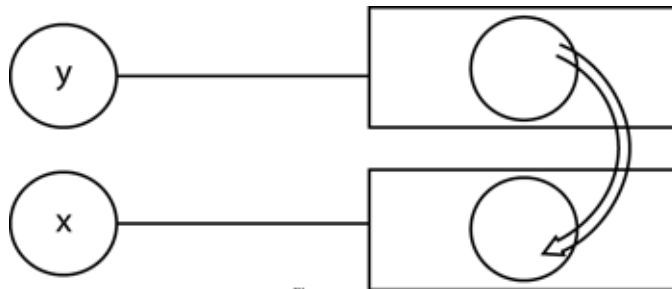
- A variable is an object whose stored value can change during execution (hence the name)
- The “change” here is commonly called “mutation” in programming languages
- Any language can potentially support mutation, but it is the procedural languages that immortalized this concept

L-value and R-value

- Variables are associated with a location and value
 - The location is called **l-value**
 - The value stored in this location is called **r-value**
- A name appearing on the left-hand side of an assignment statement must have an l-value
- A name appearing on the right-hand side must have an r-value
 - In general, an r-value is an expression

Assignment with Copy Semantics

`x = y;`



© 2003 Brooks/Cole - Thomson Learning™

- `y` is evaluated to a value which is then copied into the **location of x**
- In expression-oriented languages (C, C++, Java), an assignment expression returns a value, which is the value that was assigned to the left operand

Names in Programs

- Scope: visibility of names in source program

Static

- Storage: memory space associated with names

Dynamic

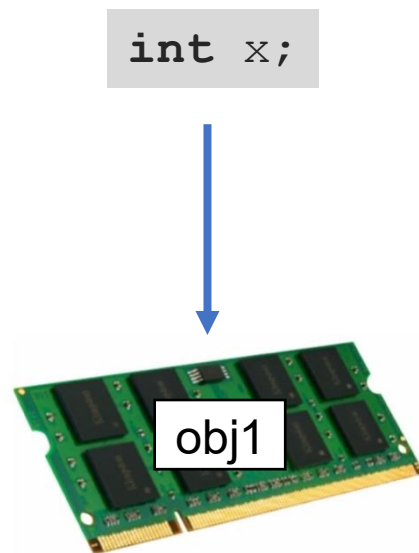
- Lifetime: the time interval a variable is allocated with memory

```
seed = DateTime.Now.Millisecond;  
number = new Random(seed);  
int maskPos = number.Next(minPos,maxPos);  
int j=0;  
do  
{  
    seed = DateTime.Now.Millisecond;  
    number = new Random(seed);  
    int newPos = number.Next(1,9);  
    int x = _setRowPosition[i]+p;  
    int y= _setColPosition[i];  
    if(_problemSet[x,y]==0)  
        +[x,y] = _nr
```



Objects and Lifetime

- Object: run-time representation of a name
- Key events of objects:
 - Creation of objects
 - Creation of bindings (references) to objects
 - (Temporary) deactivation of bindings
 - Reactivation of bindings
 - Destruction of bindings
 - Destruction of objects

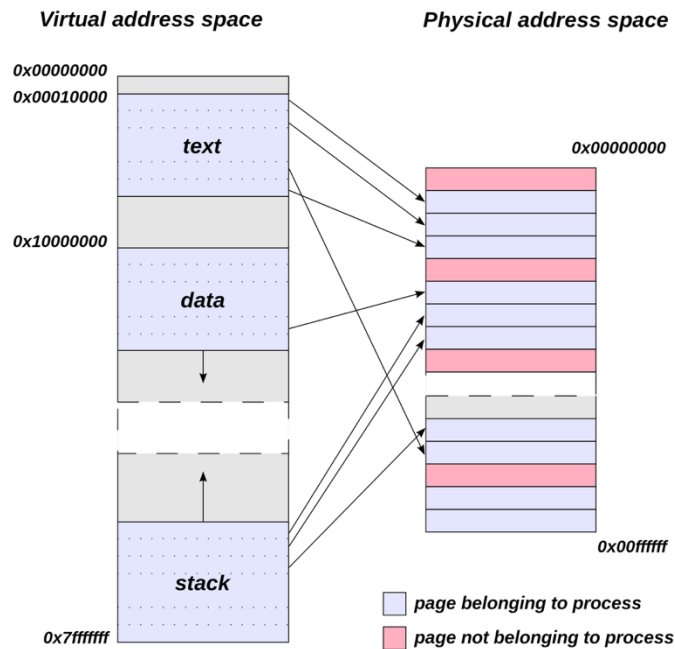
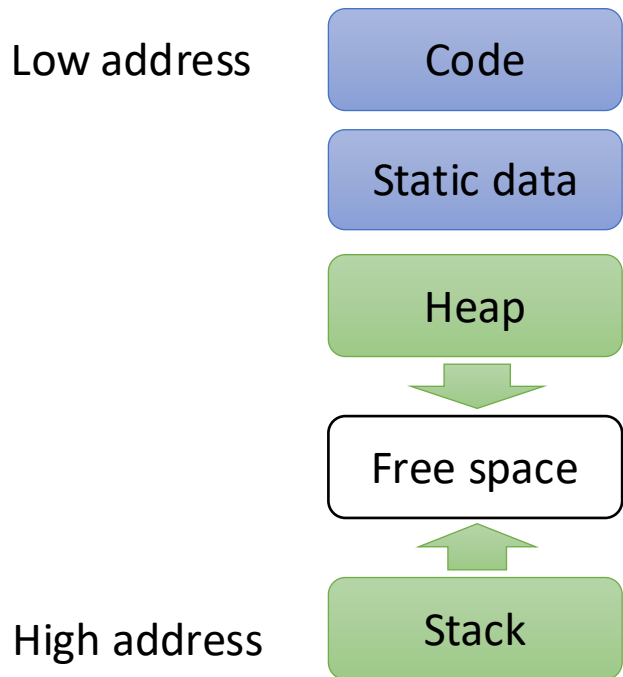




Agenda

1. Names and bindings
2. Storage allocation
3. Heap management (garbage collection)
4. Scopes

Typical Memory Layout

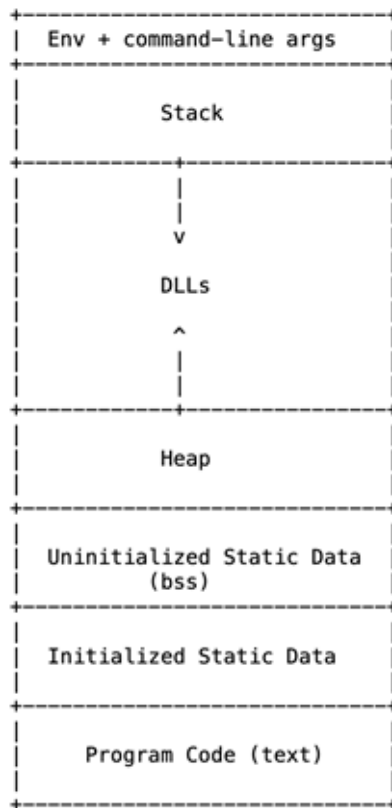


The Zoo of Allocation

- **Static allocation:** the location of allocation is decided before execution
 - Compiler translates all names to corresponding locations in the code generated by it
- **Stack allocation:** allocation/deallocation happens at procedure enter/exit time
 - Happens at run time but with details of offsets computed by the compiler (without execution)
- **Dynamic (heap) allocation:** the location of allocation is decided at runtime

The Zoo of Allocation

- Stack storage allocation
 - Static storage allocation
 - Dynamic storage allocation
- High address
- Low address





Storage Allocation & Object Lifetime

- **Static allocation**: bound to memory cells **before execution** begins and remains bound to the same memory cell **throughout execution**
- **Stack allocation**: storage bindings are created for variables when their **declaration statements are elaborated**
- **Heap allocation**: Nameless memory cells that are allocated and deallocated by **explicit directives** “run-time instructions”, specified by the programmer (either explicitly or implicitly)

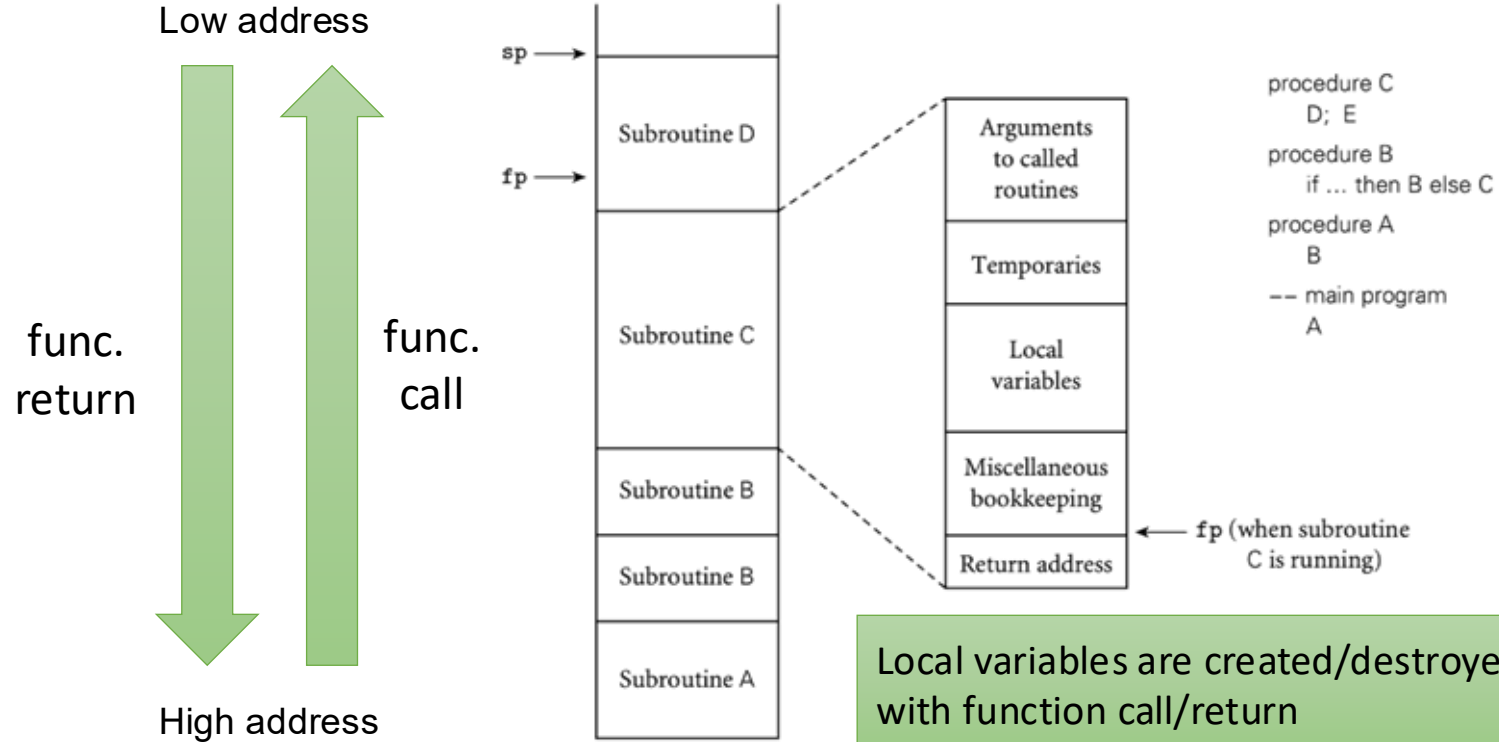
Static Allocation

- Lifetime of objects: throughout execution
- Multiple sections as static storage in an object binary file:
 - Text Section - code
 - Data Section - data
 - BSS: zero initialized or uninitialized data section
 - Data: initialized data section
 - Read-only: read-only data section
- Examples:
 - Global variables in C (In Data Section)
 - Functions (code) in C (In Text Section)
 - Static Local variables in C (In Data Section)

Stack Allocation

- In a language with procedure recursion, activations of procedure blocks cannot be allocated statically.
 - A procedure may be called again before its previous activation is exited.
- In C, all local variables and function parameters are stack-allocated

Stack Allocation



Lifetime of Local Variables

When does the lifetime of each variable begin & end?

```
int main() {  
    int x; //1  
    p(x); //2  
    q(x,x); //3  
}
```

```
int p(int p1) {  
    int px; //4  
    q(p1,px); //5  
}
```

```
int q(int q1,q2) {  
    int qx; //6  
    ...  
}
```

Dynamic (Heap) Allocation

Typically used for user-requested storage

- Allocation request (with size)
- Free request



Allocation request





Lifetime of User-Requested Storage

When does the lifetime of heap allocated objects begin & end?

```
String* x;//1
int main(){
    p();//2
    q();//3
}
```

```
int p(){
    ...
    x = new String[5];//4
    ...
}
```

```
int q(){
    ...
    delete x;//5
    ...
}
```

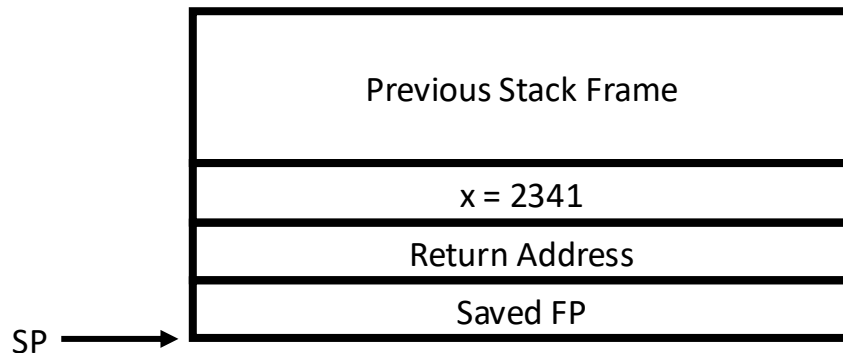
Dangling Pointers

- Locations that have been **deallocated**, but can still be accessed by a program
- What makes dangling references?
 - Pointer assignment and explicit deallocation
 - e.g. function **free** in C
 - Pointer assignment and implicit deallocation
 - by **block/procedure exit** in C

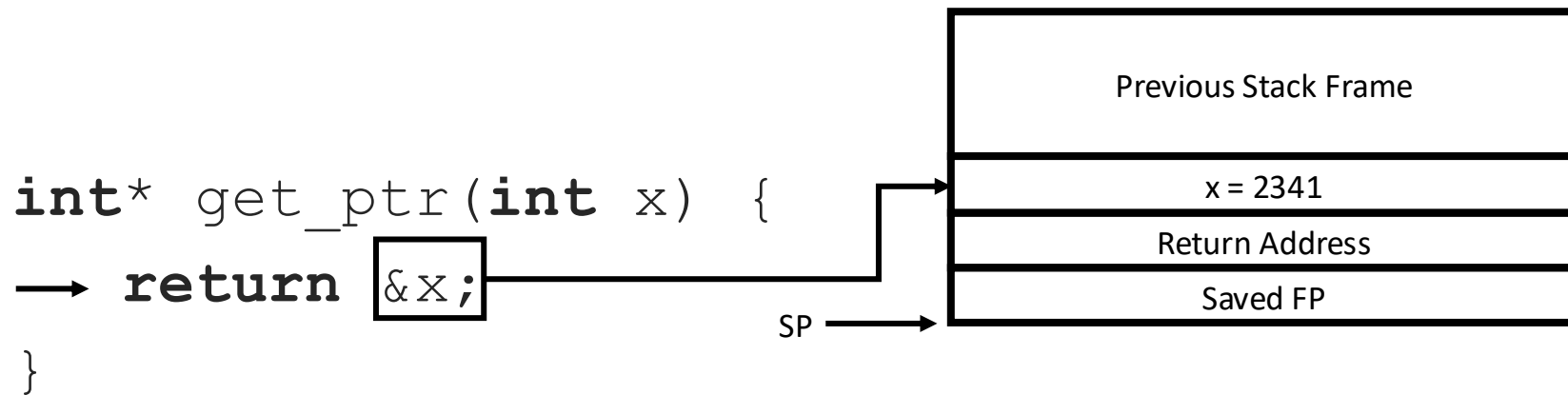


Stack Allocation and Dangling Pointers

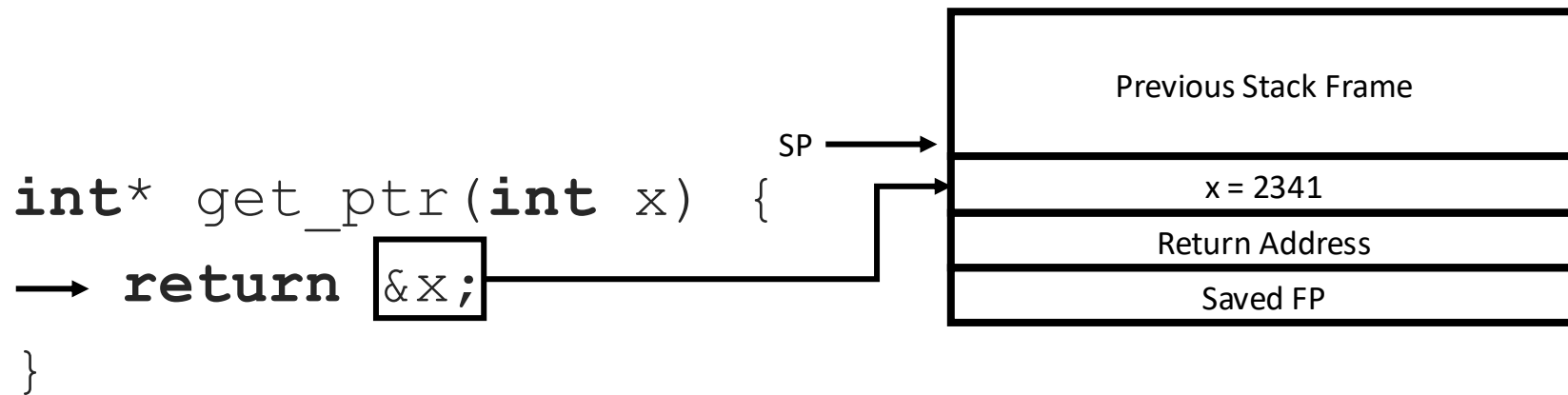
```
int* get_ptr(int x) {  
→ return &x;  
}
```



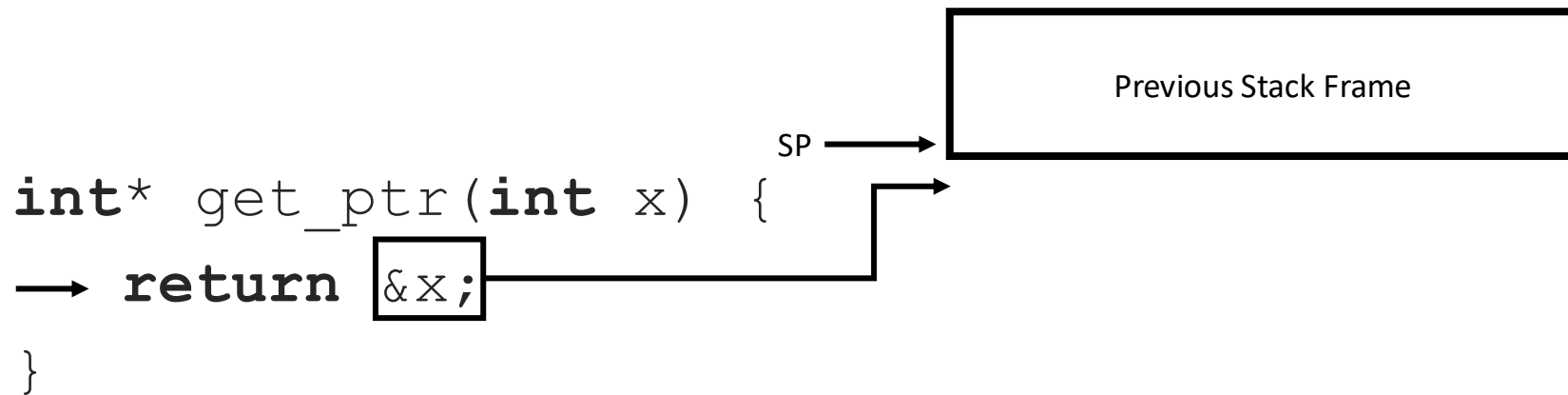
Stack Allocation and Dangling Pointers



Stack Allocation and Dangling Pointers



Stack Allocation and Dangling Pointers



Dangling Pointers with Block Exit

- Occurs if a pointer is assigned to a location that has **automatic storage management** and the lifetime of the pointer is **longer** than that of the **location**

```
{    int *x;
    {    int y;
        y = 2;
        x = &y;
    }    // x is now a dangling reference
}
```



Dangling References with Deallocation

```
main() {  
    int *x, *y;  
    x = (int *) malloc(sizeof(int));  
    *x = 2;  
    y = x;      // *y and *x are now aliases  
    free(x);    // *y now a dangling reference  
    *y = 4;  
}
```

Memory Leaks

- The issue of dangling pointers is about trying to access a location that is not allocated. The flip side of the problem is a **memory leak**
- Memory leak: when a location has been allocated, but is **no longer accessible** in a program (**garbage**)
- Memory leaks lead to the **loss of available memory**, but do not affect the correctness of programs
 - Long-running programs eventually run out of memory and crash
 - Not as serious as dangling pointer

Memory Leaks

```
void p(void) {  
    int *x;  
    x = (int *) malloc(sizeof(int));  
    x = 0;  
}
```

After `x = 0;` the memory allocated for `*x` is no longer accessible

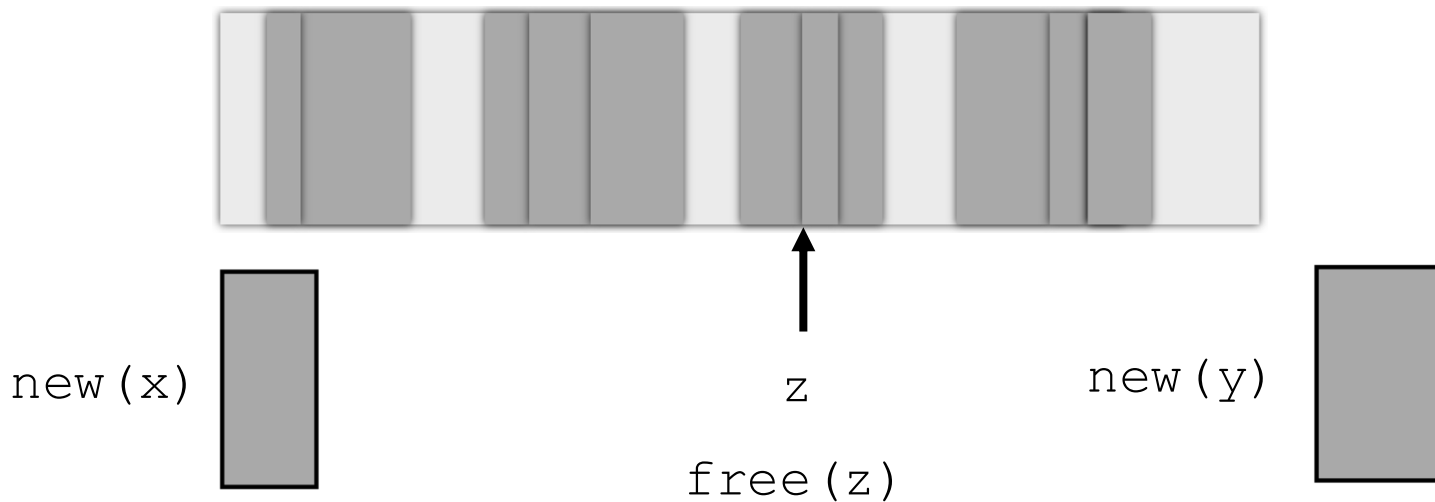


Agenda

1. Names and bindings
2. Storage allocation
3. Heap management (garbage collection)
4. Scopes

Heaps

Independent lifetimes of objects make heap management difficult



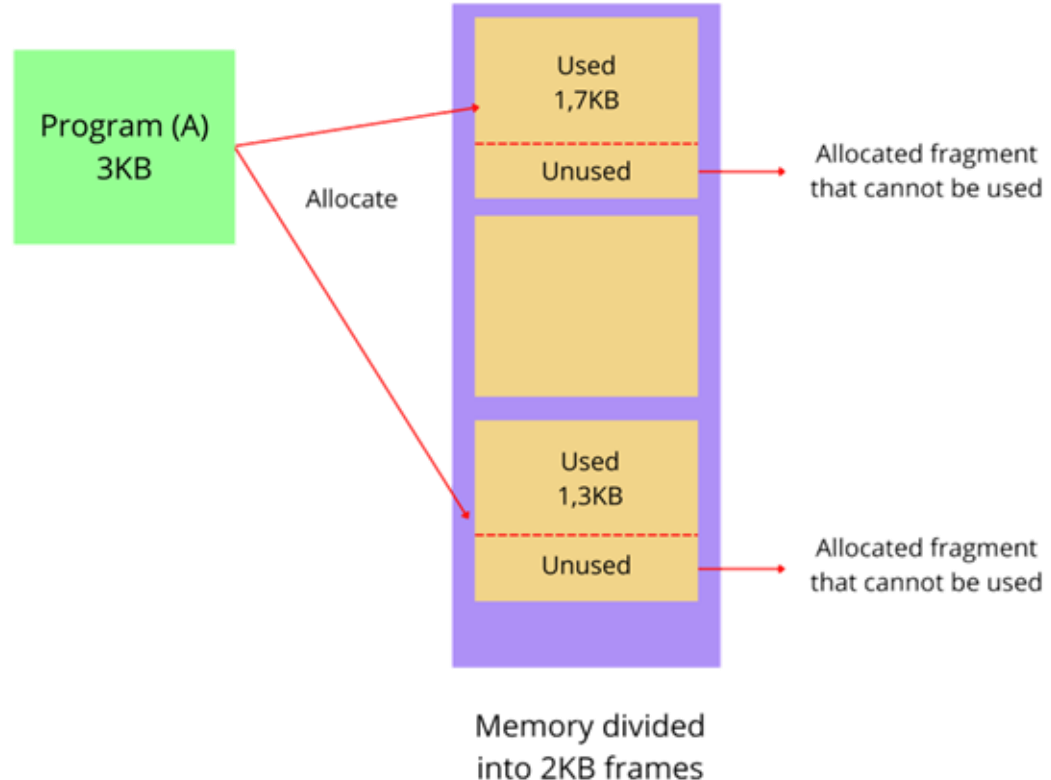
Heap Management

- Allocator: a routine takes size of requested heap space, and search for free space
 - Usually, heap is managed in blocks. Allocator may return larger block than requested
- Deallocator: collect free space and merge with other free space when possible
- Heap compaction: move all used blocks to one end

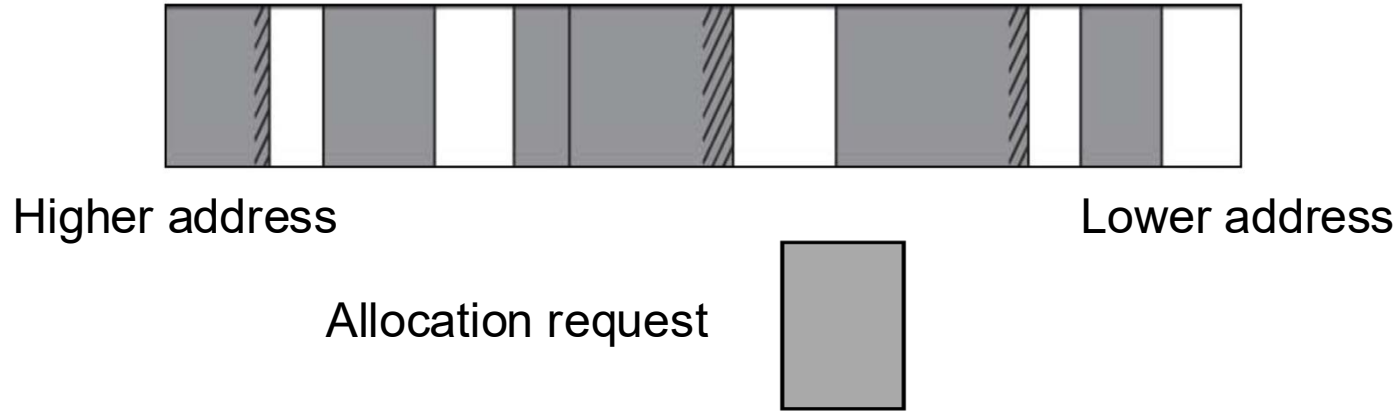


Internal fragmentation

- Occurs when the memory is distributed into fixed-sized blocks
- The allocation request is smaller than the assigned memory block



External fragmentation



- Free memory is separated into small blocks and is interspersed by allocated memory
- None of the scattered free space is large enough for the request

Heap Management Algorithms

Not covered in this course

- **First-fit**: select the first free block that is large enough
- **Best-fit**: select the smallest free block that fits
- **Buddy system**: maintain various pools of free blocks with size of 2^k
- **Fibonacci heap**: maintain various pools of free blocks with size of Fibonacci numbers

Heap Management

Programmer Management (C, C++)

- Pros: implementation simplicity, performance
- Cons: error prone (dangling pointers, memory leaks)

```
Node *p, *q;  
p = new Node ();  
q = new Node ();  
q = p; // memory leaks  
delete (p); // q becomes dangling pointer
```



Heap Management

Automatic Management (Java, Scheme)

- No dangling pointers, no memory leaks
- Cost: slower than programmer management

Garbage Collection

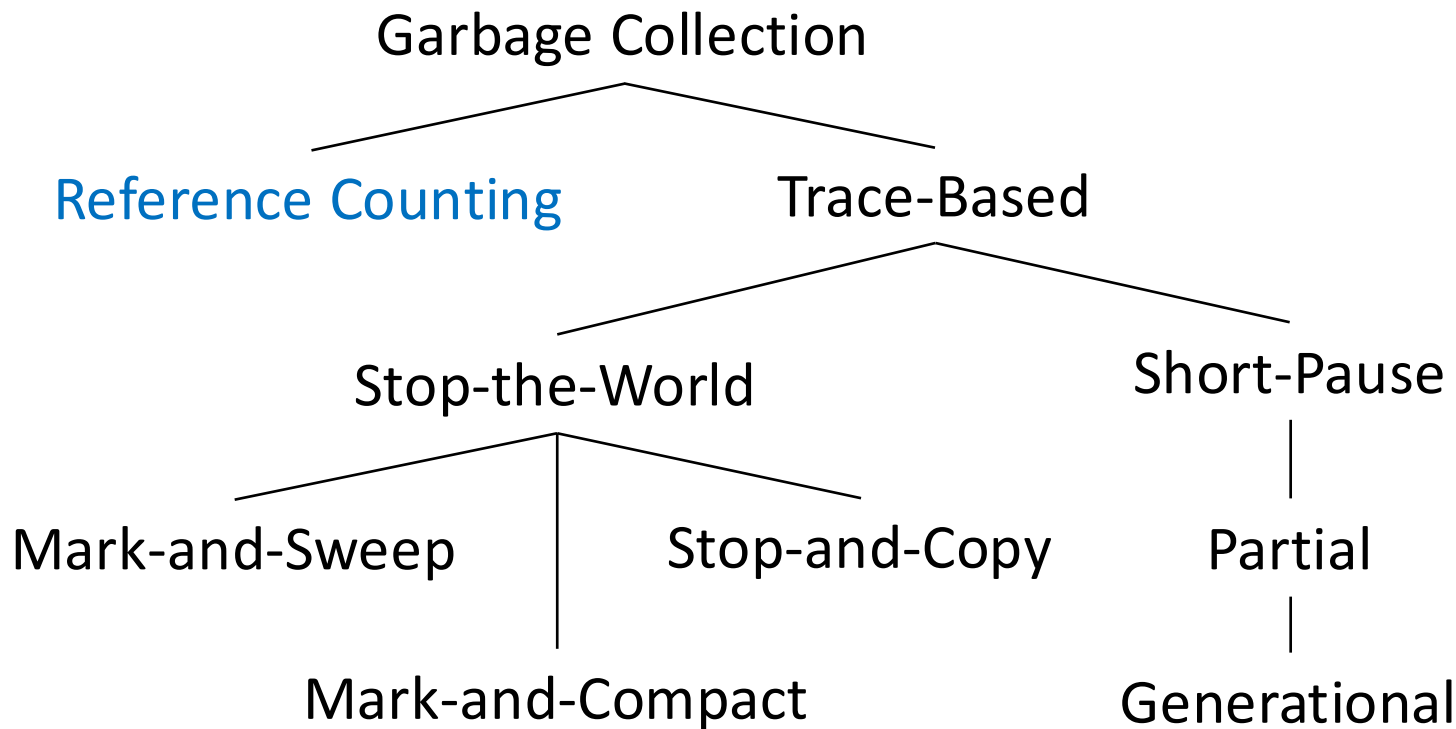
Garbage: inaccessible heap objects

```
void foo () {  
    int* a = new int[10];  
    return;  
}
```

Issues of heap management:

- Collect too aggressively: dangling pointers
- Collect too conservatively: memory leaks
- Key problem: collect only objects that are inaccessible from program

Taxonomy





GC I: Reference Counting

- Maintain a reference count with each heap object
- Set to 1 when object is created
- Incremented each time new reference to it is created
- Decrement each time reference to it is deleted
- Collect when count becomes 0

GC I: Reference Counting

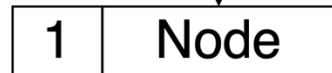
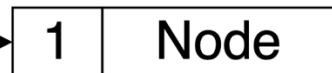
```
Node *p, *q;  
p = new Node();  
q = new Node();  
→ q.next = new Node();  
q = p;  
p = null;
```

Stack

p

q

Heap



GC I: Reference Counting

```
Node *p, *q;  
p = new Node();  
q = new Node();  
q.next = new Node();  
→ q = p;  
p = null;
```

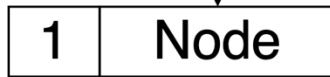
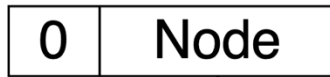
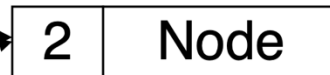
Object referenced by q has RC-1
Object referenced by p has RC+1

Stack

p

q

Heap



GC I: Reference Counting

```
Node *p, *q;  
p = new Node();  
q = new Node();  
q.next = new Node();  
q = p;  
→ p = null;
```

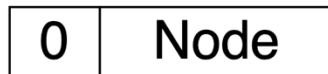
Object referenced by p has RC—1

Stack

p

q

Heap



Garbage Collection

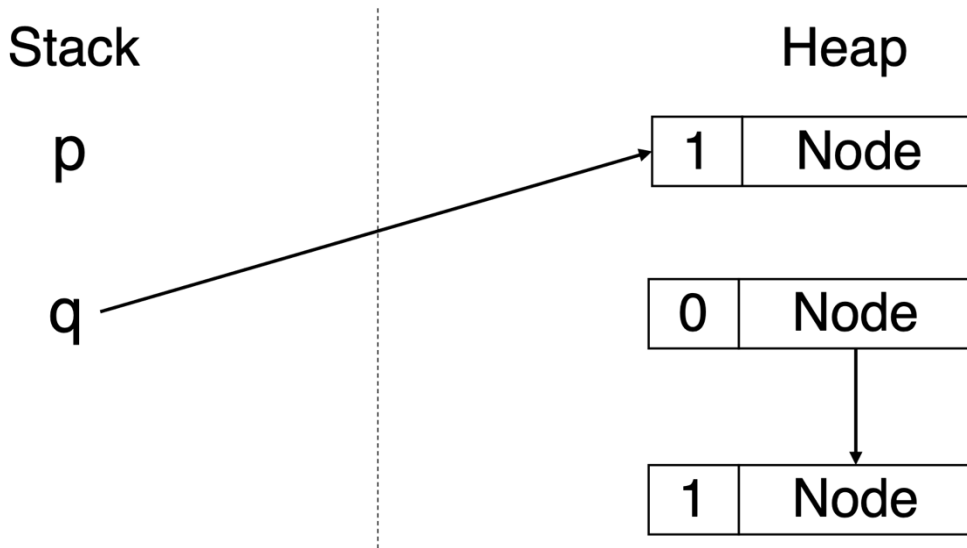
Object where $RC=0$
is collected

Stack

p

q

Heap



Garbage Collection

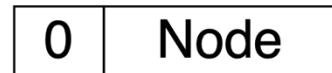
Object referenced by
collected object has
RC-1

Stack

p

q

Heap





GC I: Reference Counting

When is an object dereferenced?

- Reference is LHS of Assignment
- Reference on stack is destroyed when function returns
- Reference is destroyed when an object with count 0 is collected



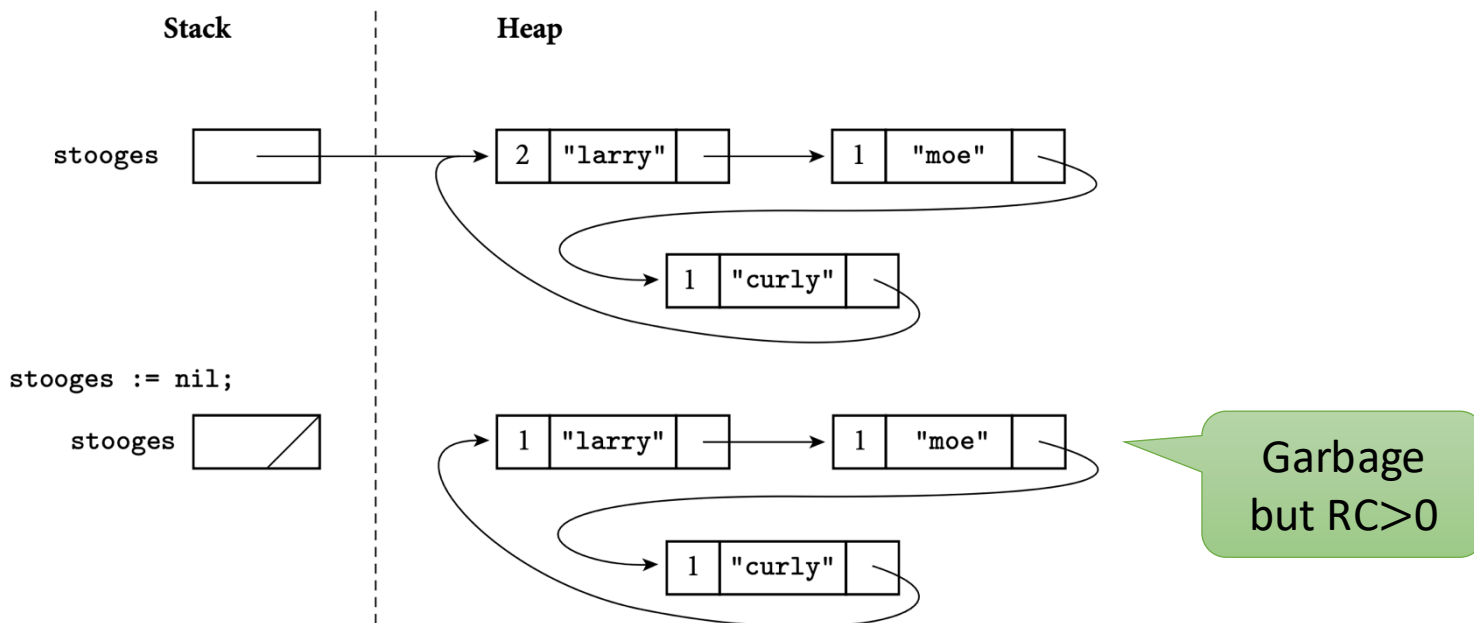
GC I: Reference Counting

Reference Counting is about **Object Ownership**

- When one object creates a reference to another object, it owns that object (retain)
- When the object deletes that reference, it relinquishes ownership (release)
 - Multiple owners of an object
 - Zero owners of an object

Problem of Reference Counting

Ownership $\neq$ Accessibility

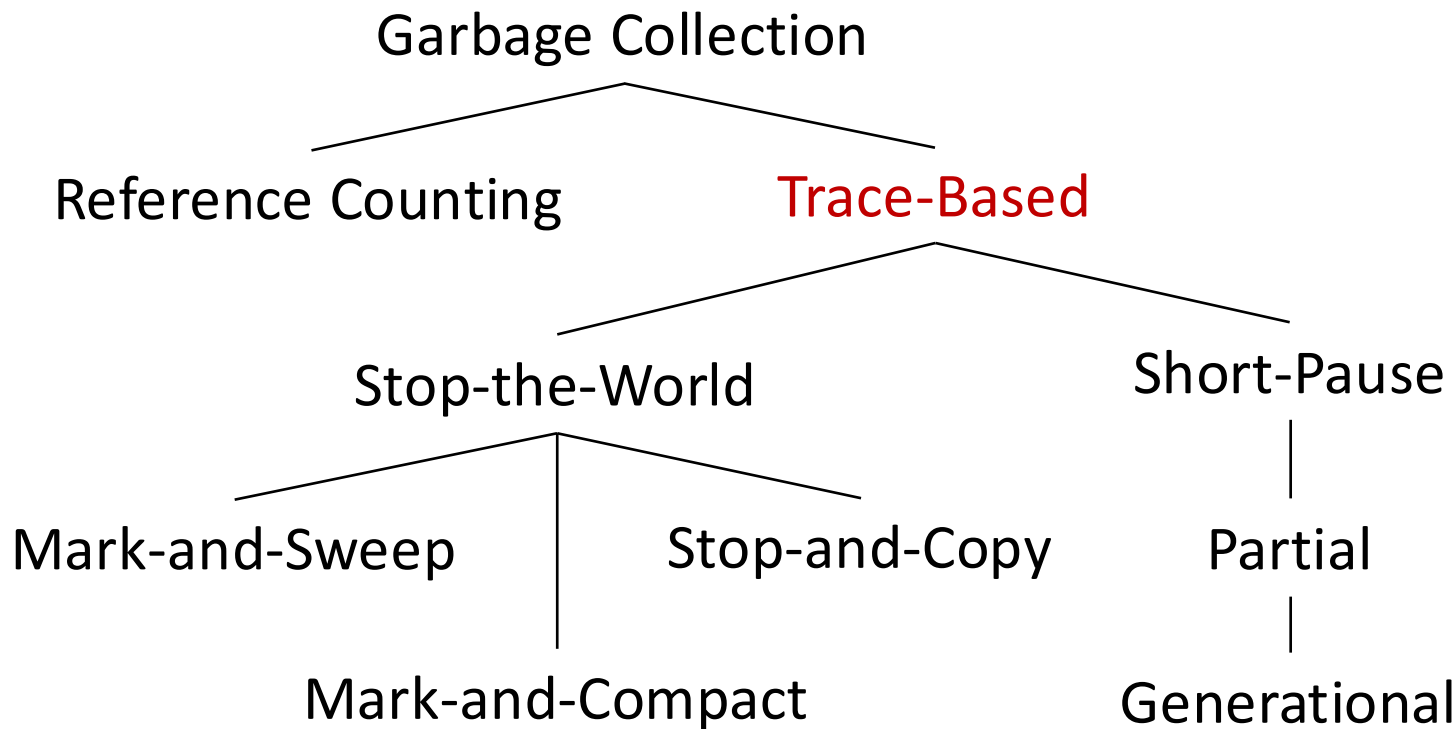




What Is Garbage

- Ideally, any heap block not used in the future
- In practice, the garbage collector identifies blocks inaccessible from program
- Essentially a **reachability** problem (from alive variables)

Taxonomy





GC II: Tracing Collection

What allocated blocks are still accessible (live)?

- Reference in registers, static area, or stack
- Reference from reachable objects to other objects

Essentially a **reachability** problem (heap as directed graph)

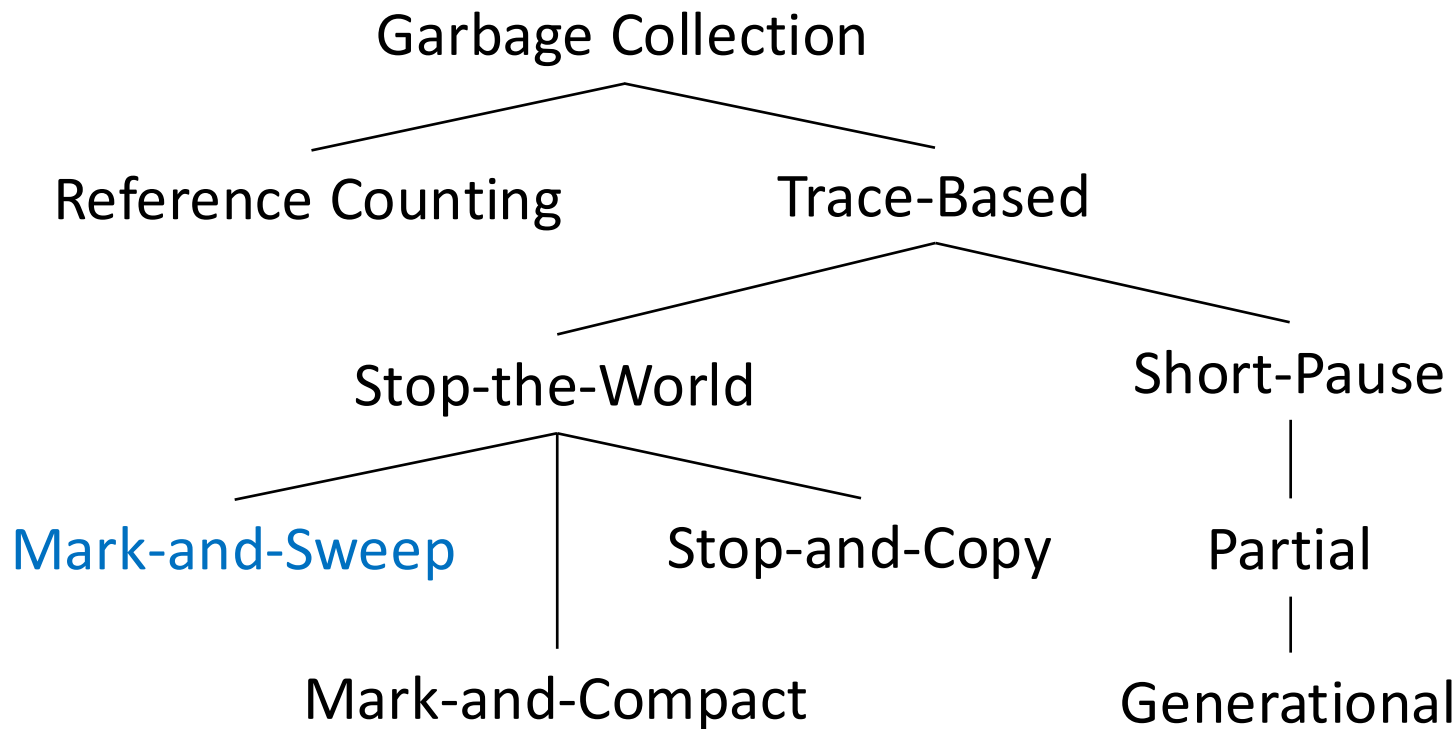
Abstraction of Tracing

- **Root set**: references in registers, static area or stack
- **Heap graph**:
 - nodes: objects
 - edges: pointers/references
- **Accessible set**: reachable objects from root set

Overview

- Start from references in registers/stack/static area
- Traverse the graph to identify reachable blocks
- Remove unreachable blocks
- Tradeoffs
 - Time overhead: % of time spent on collection
 - Space overhead: memory needed for collection algorithm
 - Pause time: the time when a program is interrupted
 - Frequency of collection
 - Promptness: time between a block becomes garbage and it is collected

Taxonomy



Mark-and-Sweep

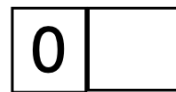
- Triggered when the **heap is full**, interrupt program
 - 1 Mark Bit (MB) for each object, initially 0
- **Phase 1 (Mark)**
 - Traverse graph, set MB to 1 for visited node
- **Phase 2 (Sweep)**
 - Traverse **all objects in heap**, collect objects whose MB=0; otherwise, set MB to 0 (prepare for the next collection)

Mark-and-Sweep Example

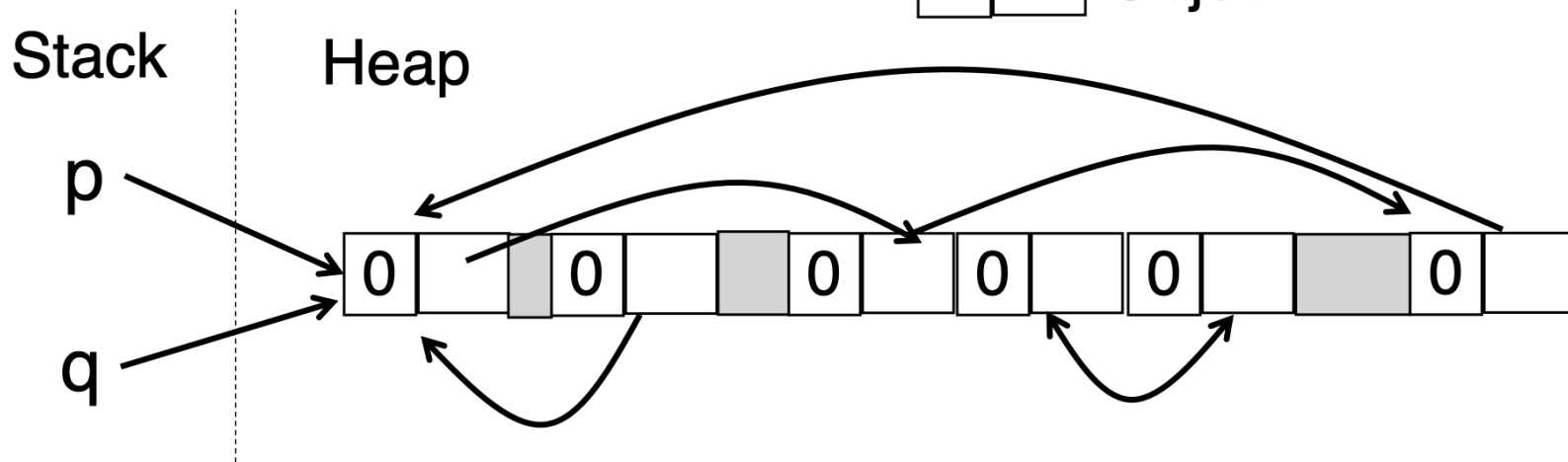
Initial state



Free space



Object with MB=0

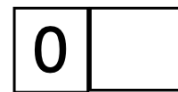


Mark-and-Sweep Example

Mark



Free space



Object with MB=0

Stack

Heap

p

q

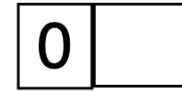


Mark-and-Sweep Example

Sweep



Free space



Object with MB=0

Stack

Heap

p

q



Mark-and-Sweep

A : number of alive objects; N : all objects on heap

- Phase 1 (Mark): $O(A)$

- Traverse graph, set MB to 1 for visited node

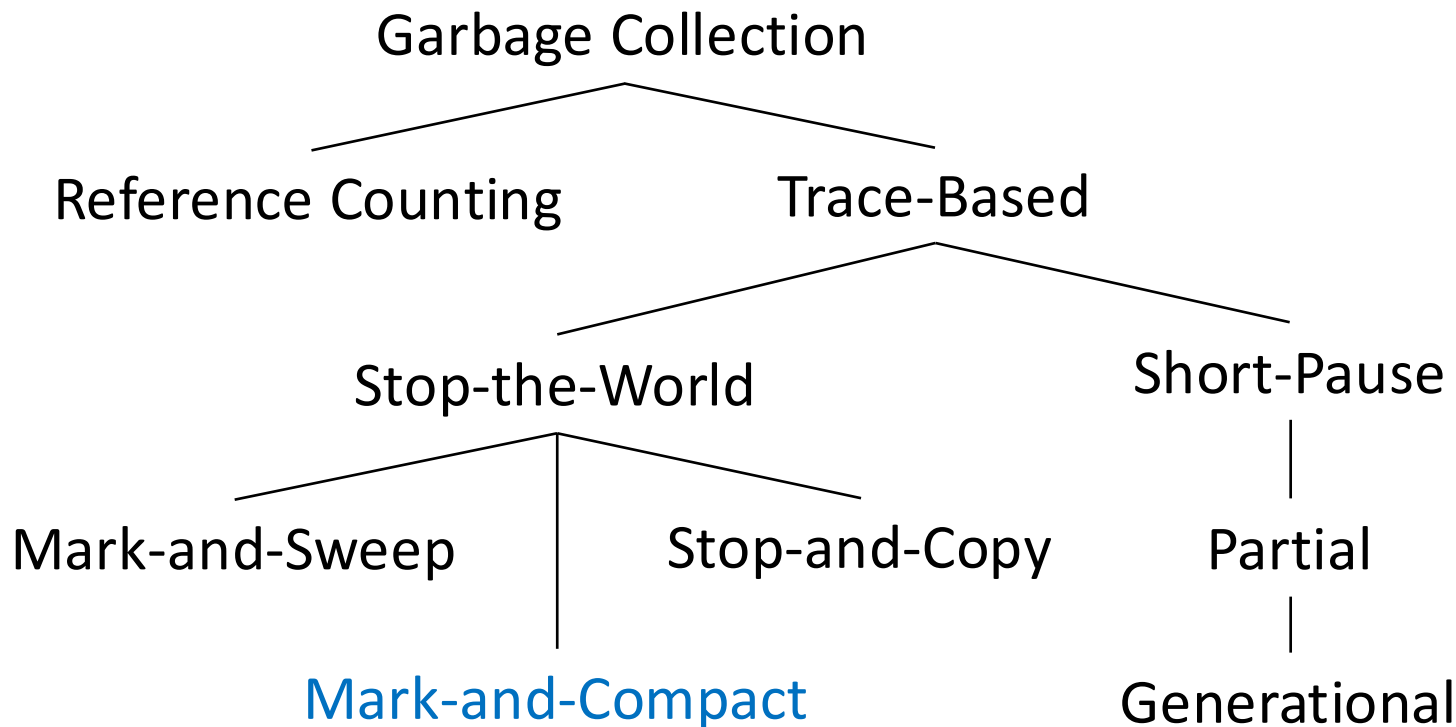
- Phase 2 (Sweep): $O(N)$

- Traverse **all objects in heap**, collect objects whose MB=0; otherwise, set MB to 0 (prepare for the next collection)

Mark-and-Sweep

- Pros:
 - Handles cycles in heap
- Cons:
 - Collection time is proportional to the heap size, since the sweep phase visits all heap objects
 - Memory fragmentation
 - Poor performance as live data grows (i.e., most objects are alive on the heap)

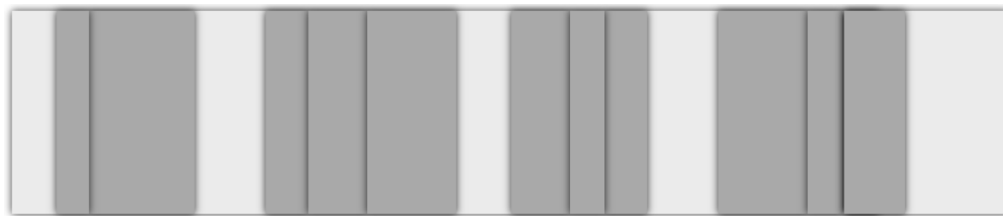
Taxonomy



Mark-and-Compact

- Similar to mark-and-sweep, except with compaction (moving reachable object to one end)

Mark-and-Sweep



Mark-and-Compact





Why Compaction

- Locality: fewer pages or cache-lines needed to hold the active data.
- Reduce fragmentation: available space merged to store large objects

Mark-and-Compact

- Triggered when the heap is full, interrupt program
- Phase 1 (Mark): same as mark-and-sweep

Mark-and-Compact

Phase 2 (Compact)

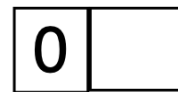
- a) Maintain
 - M: a map from object to new location
 - I: a pointer initialized to the start of heap
 - For each heap object o from low address to high address if MB=1, $M(o)=I$, $I = I + \text{size}(o)$
- b) For each heap object o from low address to high address
 - if MB=1, move o to location M(o), update pointers in o (use the map M), set MB to 0
 - if MB=0, collect o
- c) Retarget root references (use M)

Mark-and-Compact Example

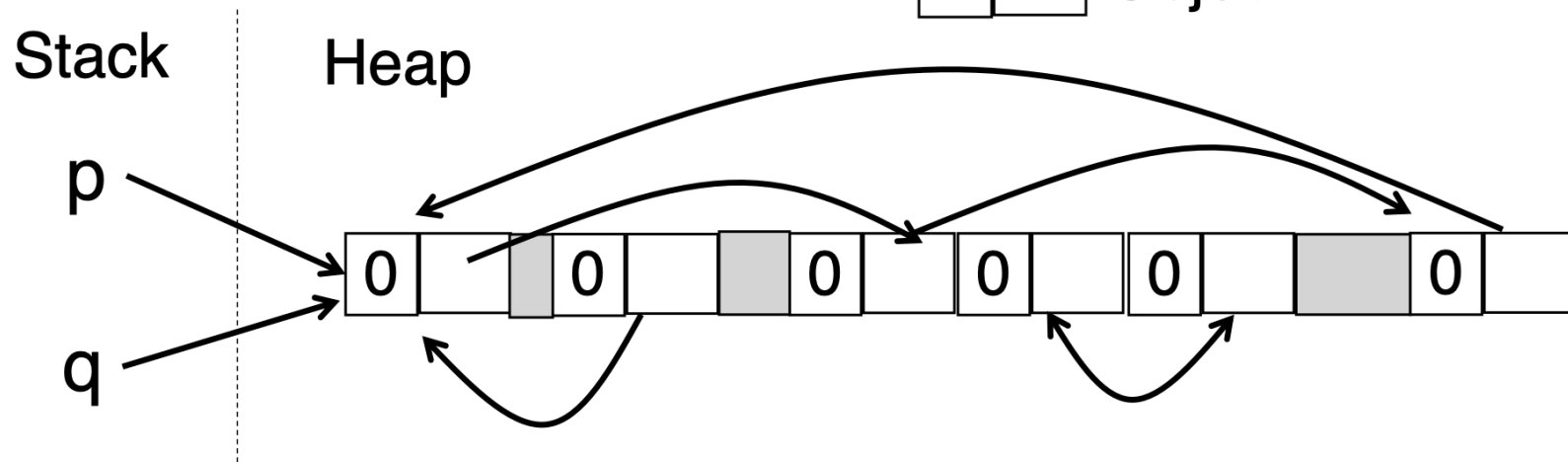
Initial state



Free space



Object with MB=0

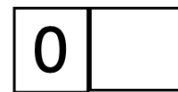


Mark-and-Compact Example

Mark



Free space



Object with MB=0

Stack

Heap

p

q



Mark-and-Compact Example

Compact: a)

 Free space

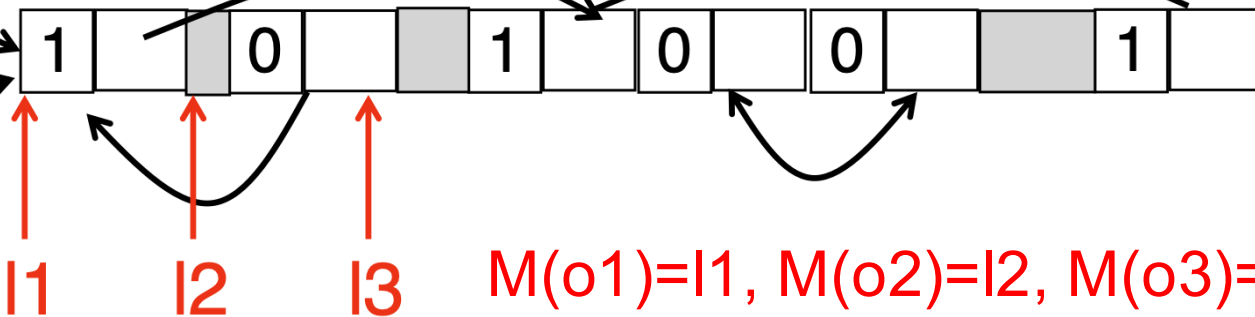
 Object with MB=0

Stack

Heap

p

q

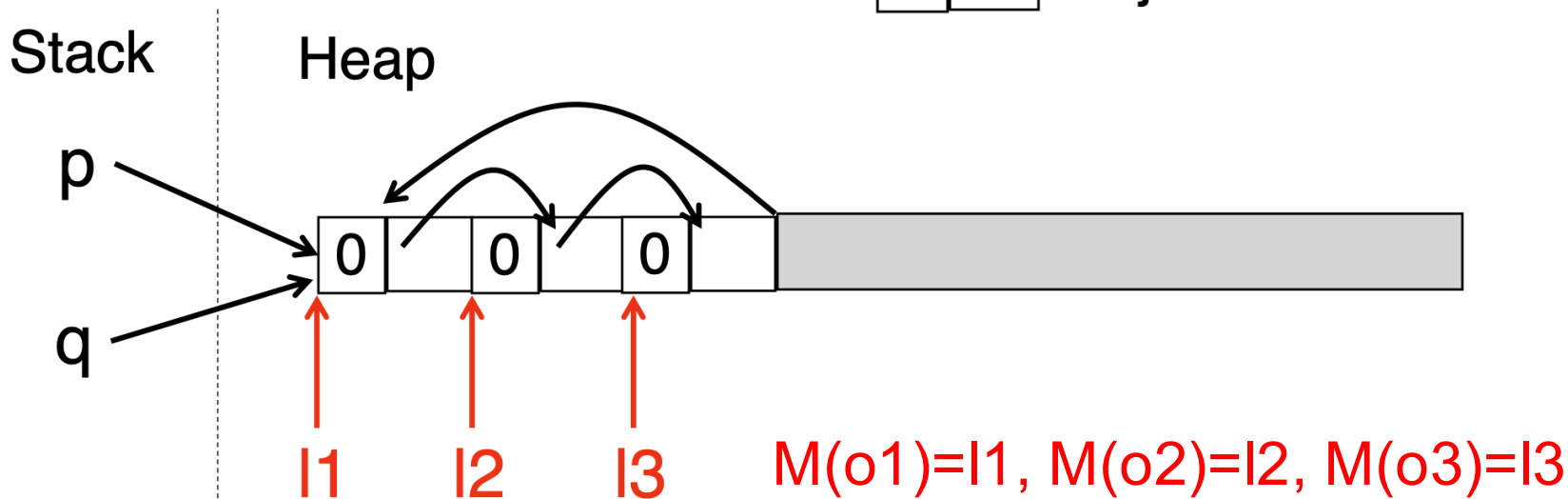


Mark-and-Compact Example

Compact: b) & c)

 Free space

 Object with MB=0



Mark-and-Compact

A : number of alive objects; N : all objects on heap

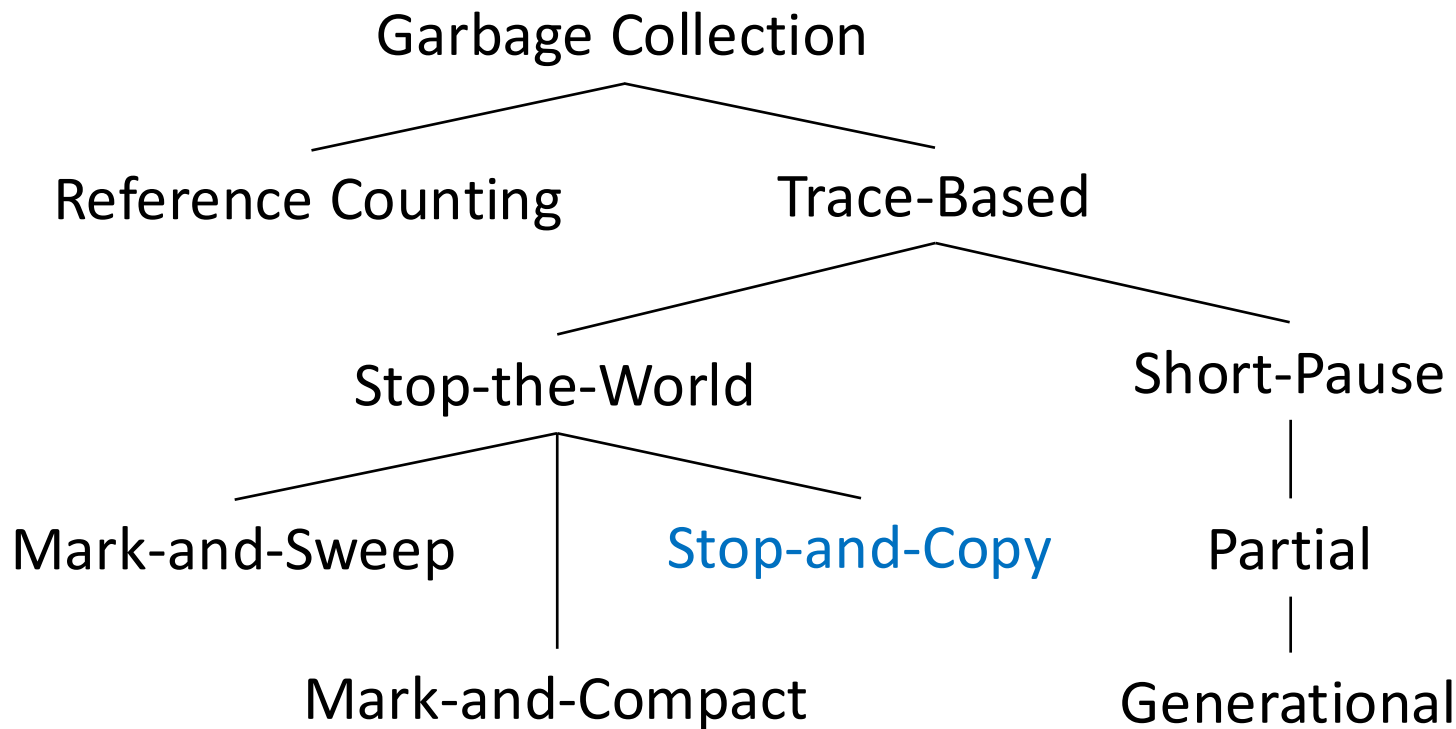
- Phase 1 (Mark): $O(A)$
 - Same as Mark-and-Sweep
- Phase 2 (Compact): $O(N)$
 - Compute new locations for objects
 - Move new objects to new location



Mark-and-Compact

- Pros:
 - Eliminates fragmentation
 - Great with long-living objects
- Cons:
 - Bad with short-lived data
 - Collection time is proportional to the heap size

Taxonomy





Stop-and-Copy (Copying Collector)

Time-Space Tradeoff:

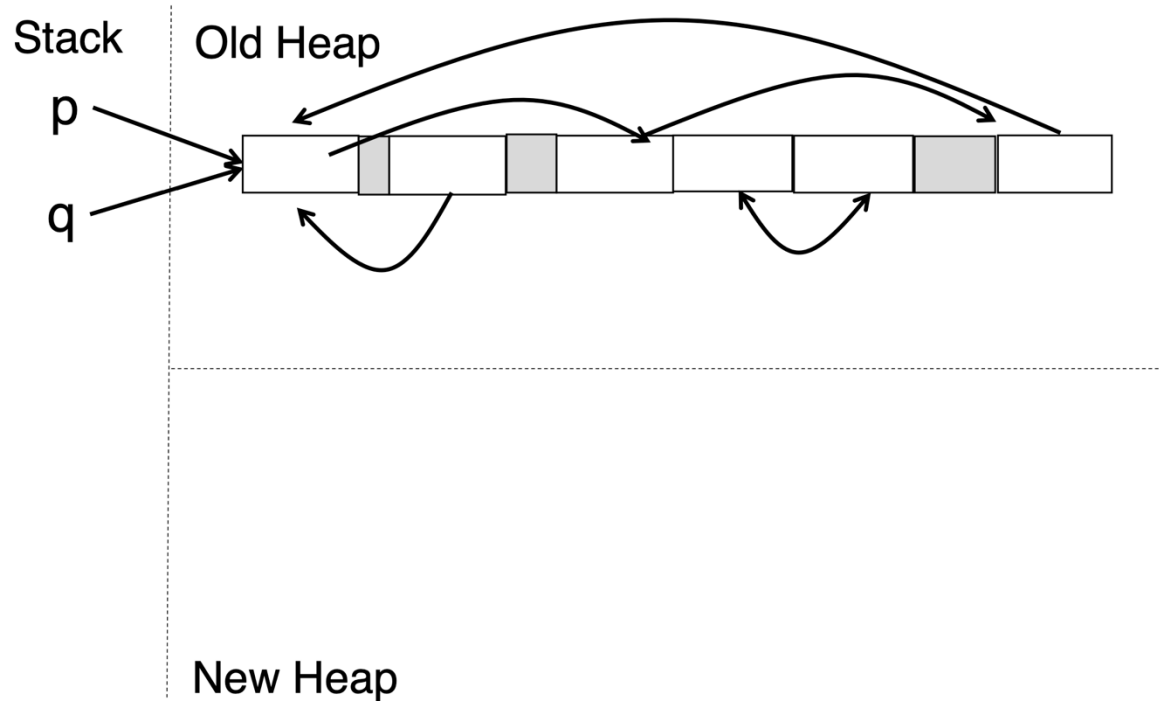
- 2 Heaps: allocate space in one, copy to second when the other is full (half of the heap is unused)
- Only **one pass over live objects** is needed (much faster than previous algorithms)
- MB bit is not needed

Stop-and-Copy

- Triggered when heap in use is full, interrupt program
- For each visited object
 1. Copy it to the other heap (no fragmentation)
 2. For the object in the old heap, keep a forwarding pointer to the new address
 3. For each object it points to:
 - if visited: update links (follow forwarding pointer)
 - otherwise, (recursively) visit object and update link
- Retarget root references
- Switch heaps

Stop-and-Copy Example

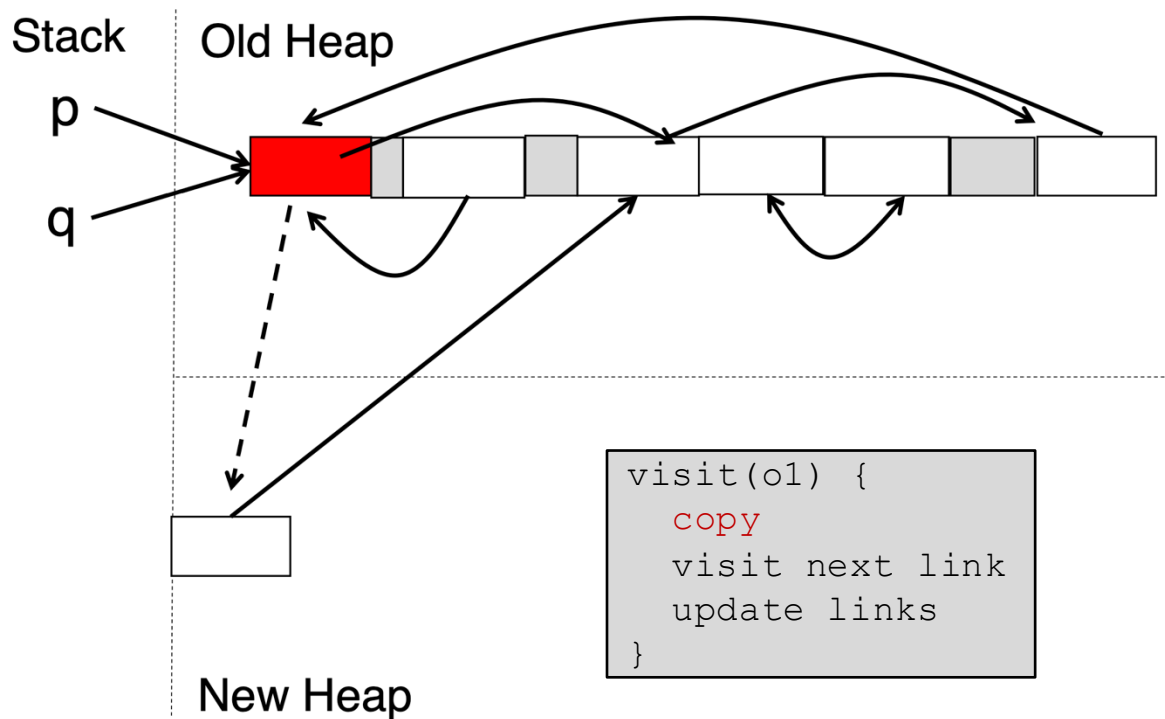
Initial state



Stop-and-Copy Example

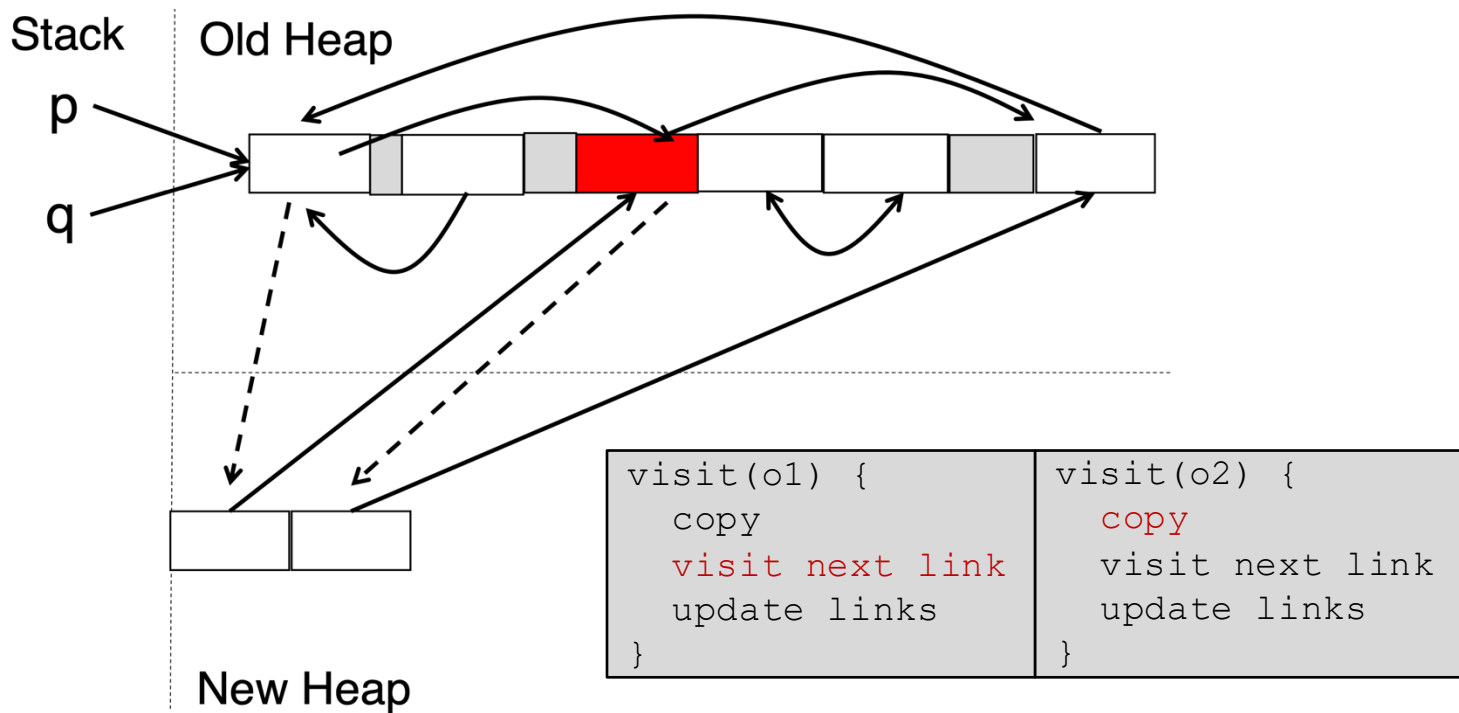
Visit red node

Forwarding Ptr:----->



Stop-and-Copy Example

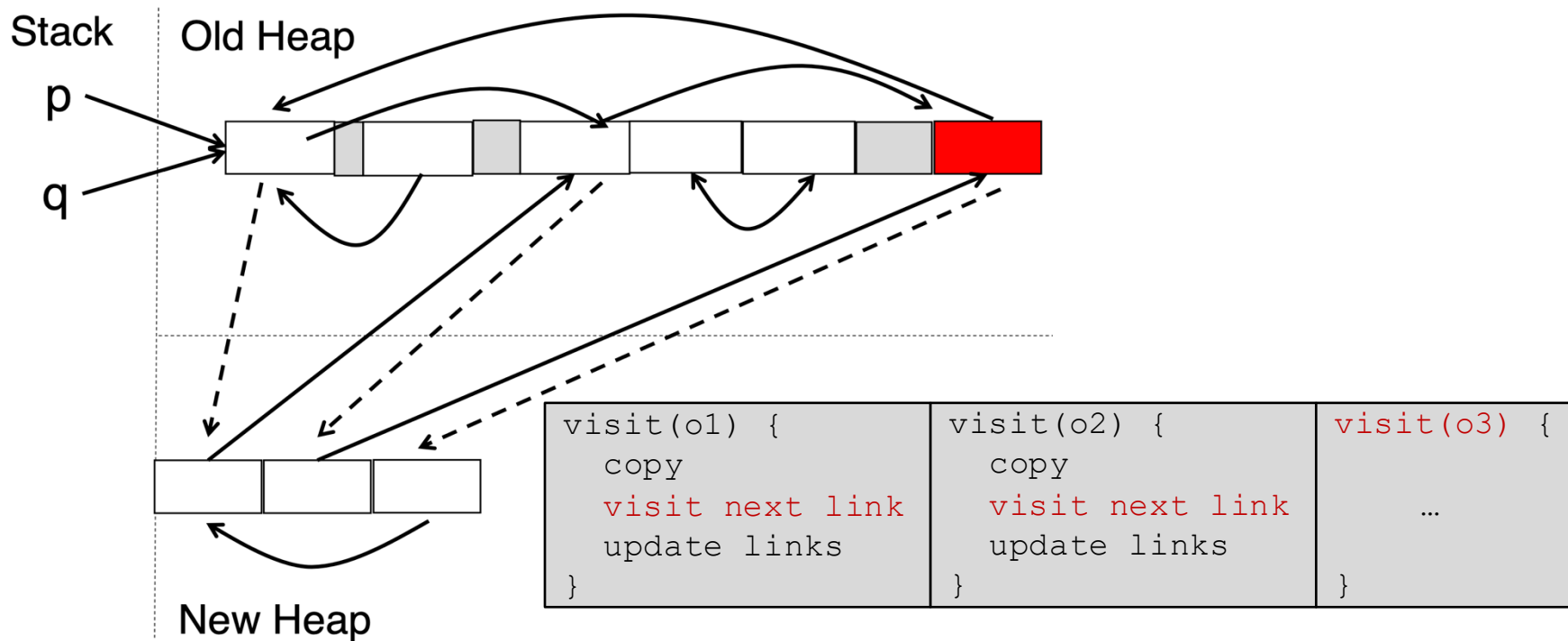
Visit red node (following link from new heap) Forwarding Ptr: ----->



Stop-and-Copy Example

Copy red node (internal link is updated)

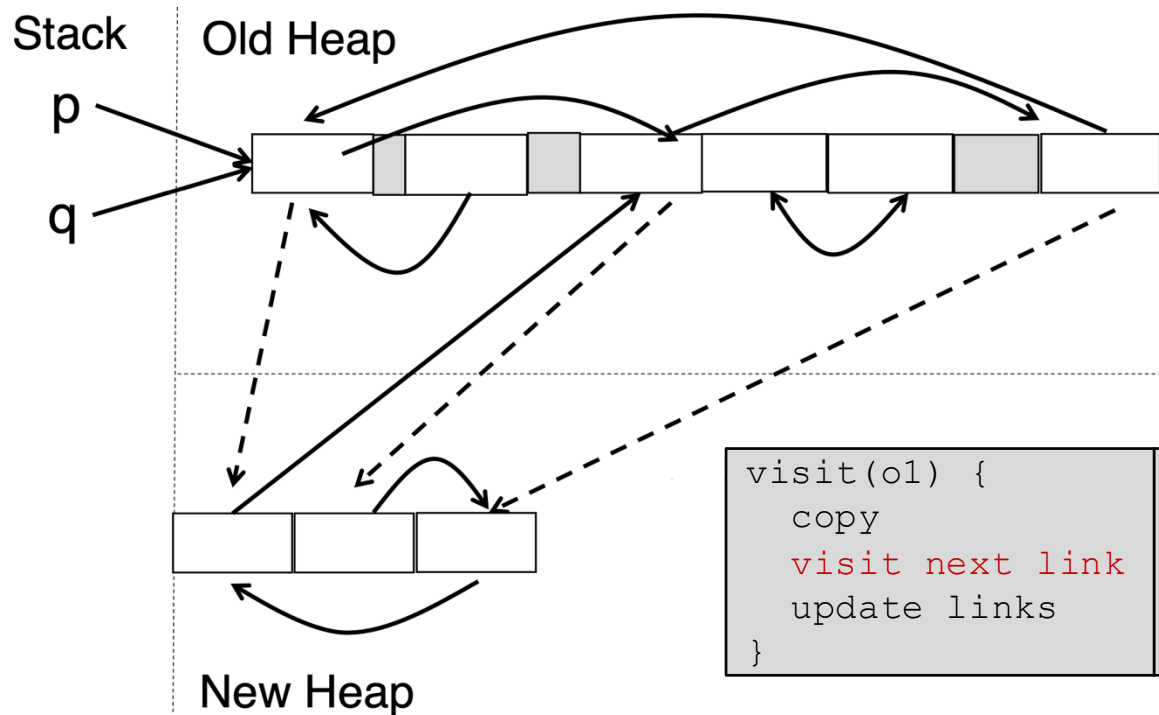
Forwarding Ptr: ----->



Stop-and-Copy Example

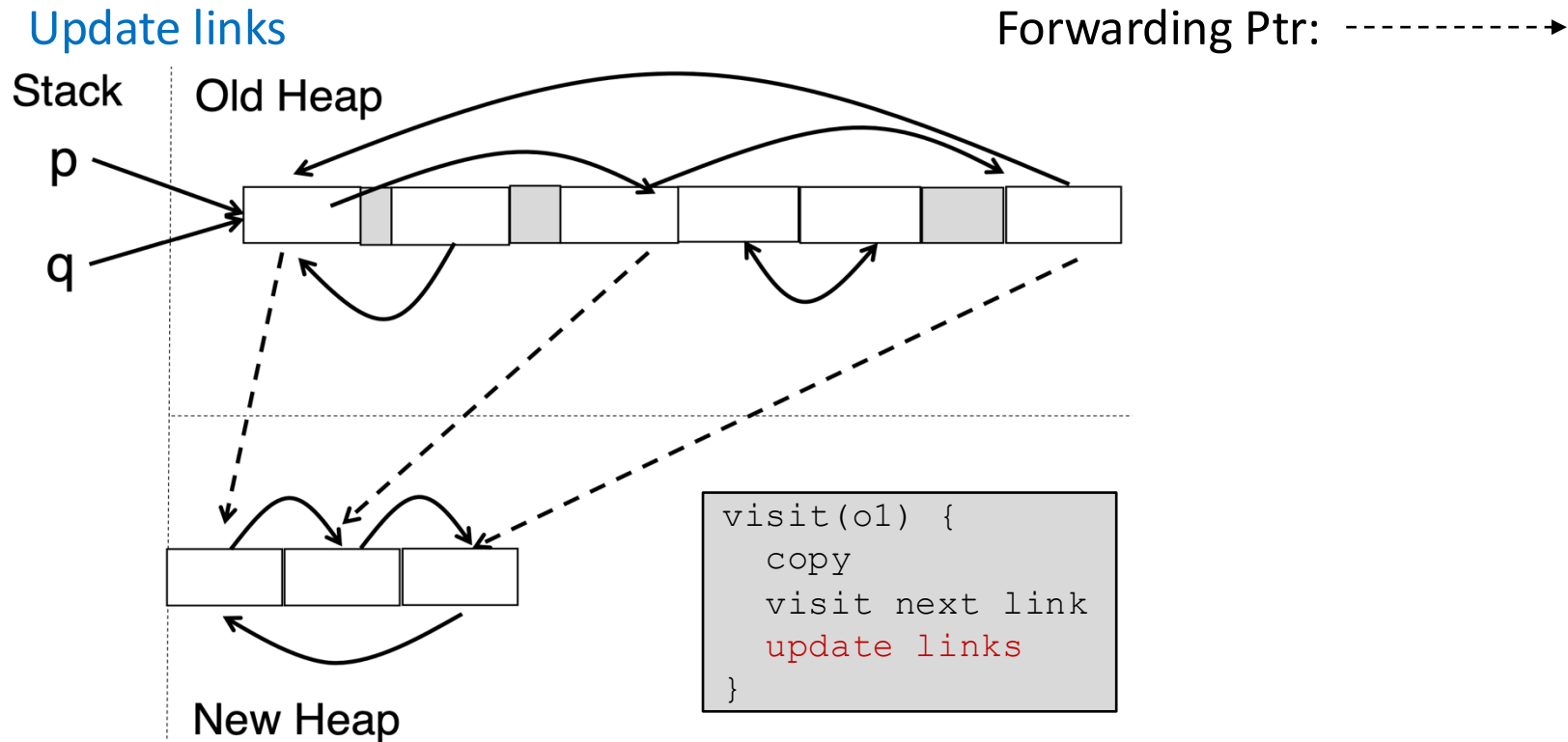
Update links

Forwarding Ptr: ----->



<pre>visit(o1) { copy visit next link update links }</pre>	<pre>visit(o2) { copy visit next link update links }</pre>
--	--

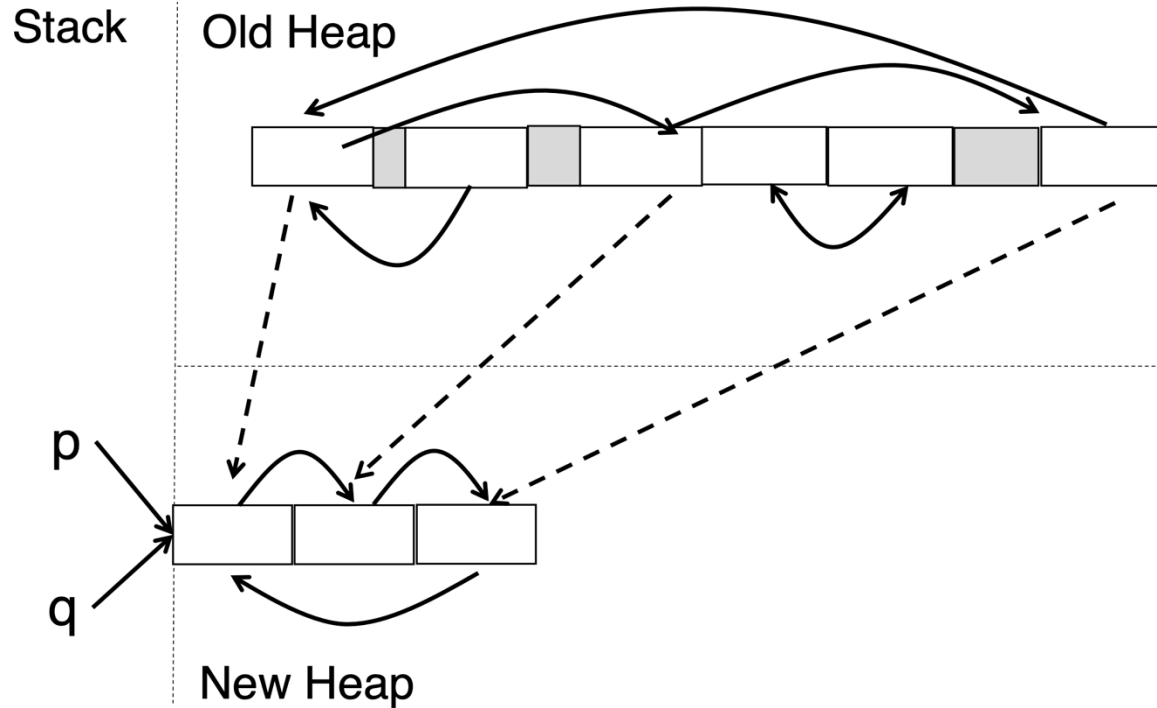
Stop-and-Copy Example



Stop-and-Copy Example

Retarget root references

Forwarding Ptr: ----->

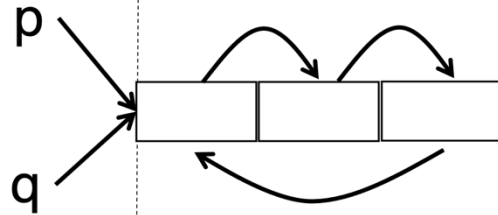


Stop-and-Copy Example

Swap Heaps (no need to clean the old one)

Stack

Heap for copying



Heap in use

Stop-and-Copy

- Pros:
 - Eliminate fragmentation
 - Collection time proportional to live objects
 - Great for short-lived data
- Cons:
 - Only half of heap is in use, requires more collecting (virtual memory alleviates this limitation)
 - Bad for long living data (copying data back and forth)

Stop-the-World Collectors

A : number of alive objects

N : all objects on heap

H : total heap size

	Mark-and-Sweep	Mark-and-Compact	Stop-and-Copy
Complexity	$O(N)$	$O(N)$	$O(A)$
Fragmentation?	Yes	No	No
Memory move	No	Yes	Yes
Effective heap size	H	H	$H/2$

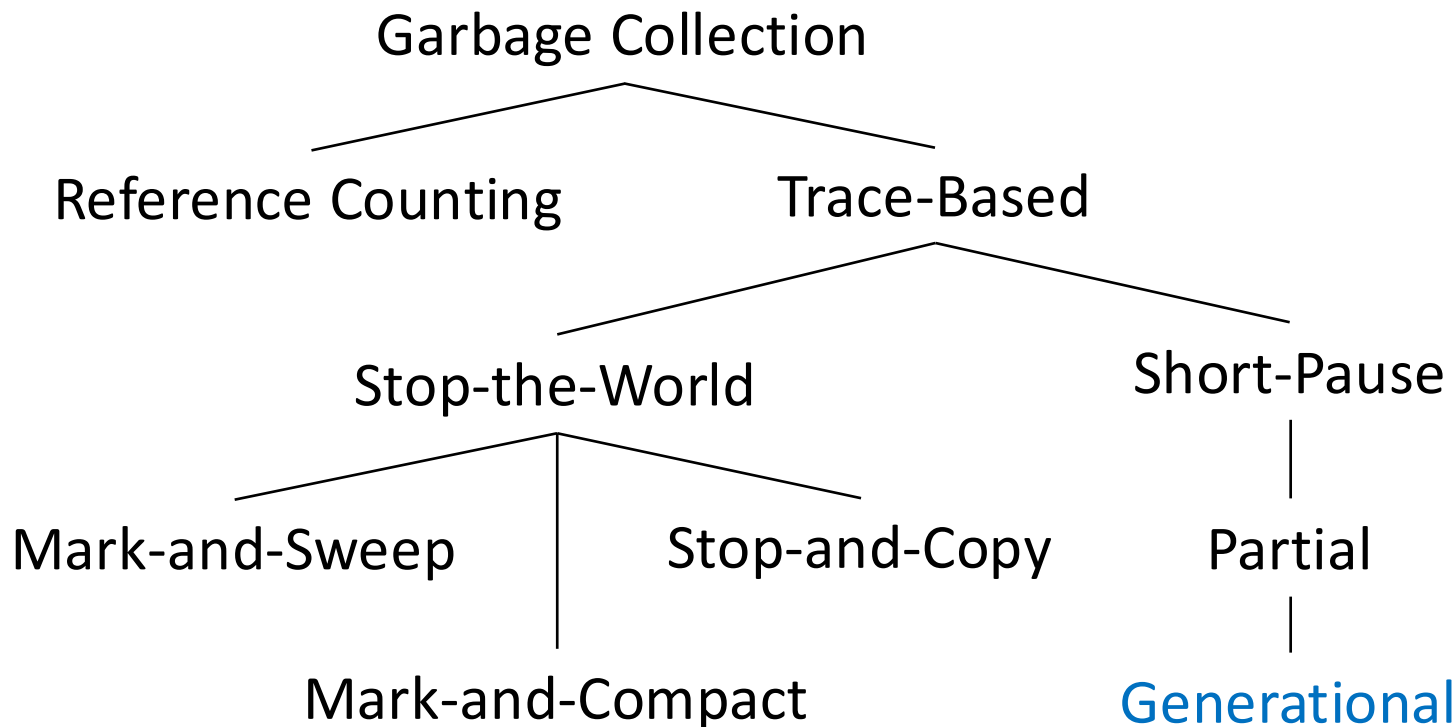


Short-Pause Collection

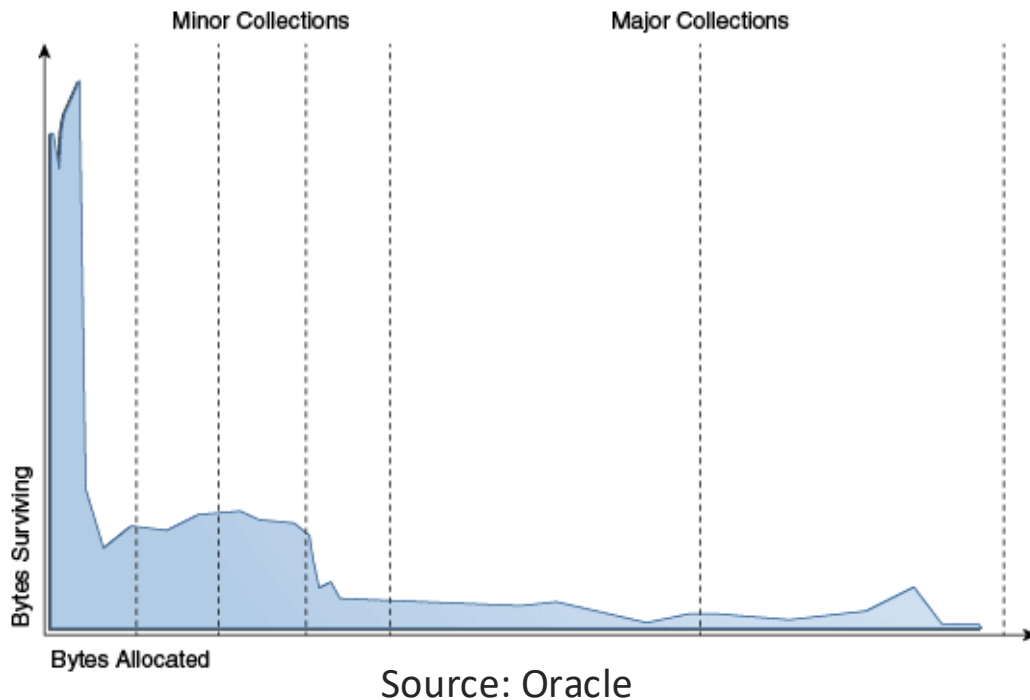
Partial

- Stop the program, but only briefly, to garbage collect **a part of** the heap

Taxonomy



Typical Distr. for Lifetimes of Objects



The Object Life-Cycle

- “Most objects die young”
- But those that survive one GC are likely to survive many
- Idea: tailor GC to spend more time on regions of the heap where objects have just been created
- A better ratio of reclaimed space per unit time

Generational

- Divide heap into **generations** $g_1, g_2 \dots$
 - g_i holds older objects than g_{i-1}
- Create new objects in g_1 , until it fills up
- GC g_1 only; move reachable objects to g_2 after several collections (typically one collection)

Generational

- When g_2 fills, garbage collect g_1 and g_2 , and put the reachable objects in g_3
- In general: When g_i fills, collect $g_1, \dots, g_i$, and put the reachable objects in g_{i+1}
- What GC algorithm is better for young generation?
How about old generation?

Algorithm for Young Generation

A : number of alive objects

N : all objects on heap

$$A \ll N$$

	Mark-and-Sweep	Mark-and-Compact	Stop-and-Copy
Complexity	$O(N)$	$O(N)$	$O(A)$
Fragmentation?	Yes	No	No
Memory move	No	Yes	Yes
Effective heap size	H	H	$H/2$

Stop-and-Copy is the most efficient

Algorithm for Old Generation

A : number of alive objects

N : all objects on heap

A is close to N

	Mark-and-Sweep	Mark-and-Compact	Stop-and-Copy
Complexity	$O(N)$	$O(N)$	$O(A)$
Fragmentation?	Yes	No	No
Memory move	No	Yes	Yes
Effective heap size	H	H	$H/2$

Mark-and-Compact removes fragmentation

Algorithm for Old Generation

A : number of alive objects

N : all objects on heap

What if most objects have the same size?

	Mark-and-Sweep	Mark-and-Compact	Stop-and-Copy
Complexity	$O(N)$	$O(N)$	$O(A)$
Fragmentation?	Yes	No	No
Memory move	No	Yes	Yes
Effective heap size	H	H	$H/2$

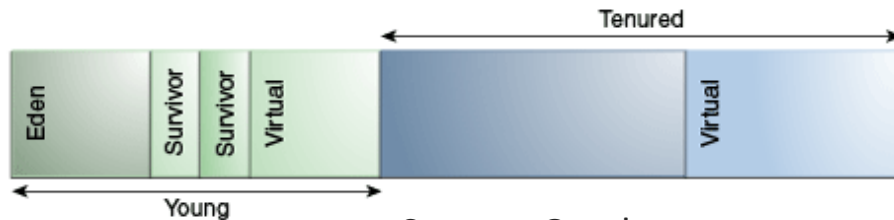
Mark-and-Sweep saves the cost of moving objects

Generational

- Pros:
 - Divide heap according to lifetimes of data
 - Great for data with mixed lifetimes
- Cons:
 - Garbage might survive a collection
 - Small heap size for each generation
 - More frequent collection

Case Study: Java SE

Generational Collectors



- Young Generation, 3 partitions:
 - Eden, 2 Survivor Spaces (stop-and-copy)
 - Minor collections (only young generation, high mortality rate)
- Old Generation
 - Surviving objects promoted from Young Generation
 - Major collections (entire heap, infrequent)
- Permanent Generation
 - Objects needed by JVM, Strings